

AIM2- Preparations

DAY 5 — Loops & Data Structures Deep Dive

Day 5 Learning Goals

Today you'll learn:

- `range()`
- `while`
- `break`
- `continue`
- `enumerate()`
- `zip()`
- Nested loops
- Tuples
- Sets
- List comprehensions
- Dictionary comprehensions
- Practical data-processing patterns

`range()`

You've already used:

```
for student in students:
```

Now let's control how many times a loop runs.

```
for number in range(5):  
    print(number)
```

Output:

0
1
2
3
4

Important:

range(5) means:

Start at 0 and stop **before 5**.

So:

range(5)

↓

0 1 2 3 4

Starting number

```
for number in range(1, 6):  
    print(number)
```

Output:

1
2
3
4
5

The structure is:

range(start, stop)

Remember:

stop is excluded.

Step

You can also specify a step:

```
for number in range(0, 11, 2):  
    print(number)
```

Output:

```
0  
2  
4  
6  
8  
10
```

Structure:

```
range(start, stop, step)
```

2 while loop

A **for** loop is generally used when you're **iterating over something**.

A **while** loop means:

Keep doing this **while a condition is true**.

Example:

```
count = 1  
  
while count <= 5:  
    print(count)  
    count += 1
```

Output:

1
2
3
4
5

Notice:

```
count += 1
```

Without that, you could accidentally create an **infinite loop**.

3 break

break means:

Stop the loop immediately.

Example:

```
for number in range(1, 11):  
  
    if number == 5:  
        break  
  
    print(number)
```

Output:

1
2
3
4

When Python reaches:

```
number == 5
```

it leaves the loop.

4 continue

continue means:

Skip this iteration and continue with the next one.

Example:

```
for number in range(1, 6):  
  
    if number == 3:  
        continue  
  
    print(number)
```

Output:

1
2
4
5

It skipped 3.

Think:

break → leave the loop
continue → skip this round

5 enumerate() ★

This is **very useful**.

Suppose:

```
subjects = [  
    "Python",  
    "Machine Learning",  
    "Deep Learning"  
]
```

You could do:

```
for subject in subjects:  
    print(subject)
```

But what if you also want the index?

You could do:

```
for index, subject in enumerate(subjects):  
    print(index, subject)
```

Output:

```
0 Python  
1 Machine Learning  
2 Deep Learning
```

You can start counting from 1:

```
for index, subject in enumerate(subjects, start=1):  
    print(index, subject)
```

Output:

```
1 Python  
2 Machine Learning  
3 Deep Learning
```

🔥 Remember `enumerate()`. You'll use it often.

6 zip() ★

Suppose:

```
names = ["Krishna", "Rahul", "Priya"]  
scores = [90, 85, 95]
```

You want:

```
Krishna → 90  
Rahul   → 85  
Priya   → 95
```

Use:

```
for name, score in zip(names, scores):  
    print(name, "->", score)
```

Output:

```
Krishna -> 90  
Rahul   -> 85  
Priya   -> 95
```

This is extremely useful when working with related datasets.

7 Nested loops

A loop inside another loop.

Example:

```
students = ["Krishna", "Rahul"]  
subjects = ["Math", "Python"]  
  
for student in students:
```

```
for subject in subjects:  
    print(student, "-", subject)
```

Output:

```
Krishna - Math  
Krishna - Python  
Rahul - Math  
Rahul - Python
```

Don't overuse nested loops, but you need to understand them.

Tuples

A tuple looks like a list:

```
student = ("Krishna", 28, 95)
```

Access:

```
print(student[0])
```

Output:

```
Krishna
```

The major difference:

List

```
student = ["Krishna", 28, 95]
```

Can be changed.

Tuple

```
student = ("Krishna", 28, 95)
```

Cannot be changed after creation.

For example:

```
student[0] = "Rahul"
```

would fail for a tuple.

Think:

List → mutable

Tuple → immutable

You'll encounter tuples quite often in Python libraries.

Sets

A set stores **unique values**.

```
numbers = {1, 2, 3, 3, 4, 4, 5}
```

```
print(numbers)
```

You'll get something equivalent to:

```
{1, 2, 3, 4, 5}
```

Duplicates disappear.

You can add:

```
numbers.add(6)
```

Remove:

```
numbers.remove(3)
```

Sets are particularly useful when you care about **uniqueness**.

10 List Comprehension ★ ★ ★

You saw this in our original AIM2 preparation plan:

```
numbers = [10, 20, 30, 40]

squared = [x**2 for x in numbers]
```

Output:

```
[100, 400, 900, 1600]
```

This:

```
squared = [x**2 for x in numbers]
```

is basically a shorter version of:

```
squared = []

for x in numbers:
    squared.append(x**2)
```

Both work.

Why learn comprehension?

Because you'll see this style frequently in Python data-processing code.

1 1 Conditional list comprehension

You can also add a condition.

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_numbers = [x for x in numbers if x % 2 == 0]

print(even_numbers)
```

Output:

```
[2, 4, 6]
```

Conceptually:

Take x

↓

from numbers

↓

if x is even

↓

put x into new list

1 2 Dictionary comprehension

You can do the same thing with dictionaries.

```
numbers = [1, 2, 3, 4]
```

```
squares = {x: x**2 for x in numbers}
```

```
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16}
```

Don't worry about becoming an expert at comprehensions today.

Your goal is:

Understand them when you see them.



DAY 5 PRACTICE

Create:

```
week-01-python/  
└─ exercises/  
    └─ day-05-loops-data-structures.py
```

Exercise 1

Print numbers 1–20 using `range()`.

Then print only even numbers.

Exercise 2

Use a `while` loop to print:

```
5  
4  
3  
2  
1
```

Exercise 3

Use `enumerate()`:

```
subjects = [  
    "Python",  
    "Machine Learning",  
    "Deep Learning",  
    "NLP",
```

```
    "Data Mining"  
]
```

Expected:

```
1 -> Python  
2 -> Machine Learning  
3 -> Deep Learning  
4 -> NLP  
5 -> Data Mining
```

Exercise 4

Use zip():

```
students = ["Krishna", "Rahul", "Priya"]  
scores = [82.67, 83.67, 91.00]
```

Expected:

```
Krishna -> 82.67  
Rahul -> 83.67  
Priya -> 91.00
```

Exercise 5

Create:

```
numbers = [10, 15, 20, 25, 30, 35, 40]
```

Create a new list containing only even numbers using **list comprehension**.

DAY 5 MINI PROJECT

Create a **Student Statistics Analyzer**.

Use:

```
students = [  
    {"name": "Krishna", "score": 82.67},  
    {"name": "Rahul", "score": 83.67},  
    {"name": "Priya", "score": 91.00},  
    {"name": "Amit", "score": 76.50},  
    {"name": "Neha", "score": 88.25}  
]
```

Your program should calculate:

- Number of students
- Highest score
- Lowest score
- Average score
- Students who passed
- Students who failed
- List of students with score ≥ 80

DAY 5 CHECKPOINT

Before moving to Day 6, you should understand:

range()	✓
while	✓
break	✓
continue	✓
enumerate()	✓
zip()	✓
nested loops	✓
tuple	✓
set	✓

list comprehension 

dictionary comprehension → basic understanding